

Roadmap de uso del harness (OpenCode)

Cómo se usó el asistente de código en este proyecto. Cada trabajo sigue el ciclo **prompt · respuesta del asistente · evidencia verificada por mí**.

Método

- La bitácora completa **prompt·respuesta·evidencia** vive en **docs/bitacora-ia/** (16 archivos + PDFs en **docs/pdf/**).
- Nada se acepta por palabra: tests, evaluaciones y comandos se re-ejecutan antes de dar por hecho (verificación de a un comando).
- Decisiones de arquitectura y fracasos se guardan en memoria persistente (Engram) para no repetir errores.

Fases ejecutadas

- 1 **Rama limpia:** eliminada V1 legacy, tag de respaldo. Evidencia: **slice-01**.
- 2 **Infra VM + DB:** PostgreSQL 15, restore, sync 42 atenciones, índice 2886 docs **slice-02**.
- 3 **Motor LLM:** llama.cpp CUDA + Qwen2.5-3B, API key, systemd. **slice-03, slice-07**.
- 4 **RAG medido:** normalización de embeddings + umbral 0.5 calibrados **slice-04**.
- 5 **Chat y guardrails:** burbuja, 3 capas anti-jailbreak, loop de aprendizaje, panel, MCP, poderes con confirmación **slice-05·slice-12**.
- 6 **Fine-tuning honesto:** 3 noches 5/30, gate frenó despliegue. **slice-13**.
- 7 **Producción** merge, deploy vivo, evaluar 9/10, VMDK, main unificado. **slice-14**.
- 8 **Revisión nativa** 3 candidatos excedieron presupuesto · evidencia sustituta **slice-15**.

Fases de mejora de revisión (en curso)

- **Fase 1** (hecha): clasificador de riesgo por contenido (backend-fastapi/app/services/review_risk.py, 6 tests). Spec: docs/specs/revision-local-fase1.md.
- **Fase 2:** revisión por slices con presupuesto fijo.
- **Fase 3:** lens local con el propio modelo (titular de tesis).
- **Fase 4** (hecha): loop de falsos positivos (review_feedback.py, 6 tests). Spec: docs/specs/revision-local-fase4.md.